

1. (2 Punkte) Der nächste Quicksort-Durchgang sortiert die Liste von 0-2. Der nächste wartende Abschnitt ist von 3-7.

8 8 4 88 13 9 38 15

Schreibe die Protokollzeilen vor und nach dem Durchgang.

Lösung:

```
8 8 4 88 13 9 38 15 unten=0 oben=2 pivot=8
4 8 8 88 13 9 38 15 unten=3 oben=7 pivot=9
```

2. (2 Punkte) Mache aus der Liste einen Heap nach dem Verfahren aus dem Unterricht.

14 2 33 89 56 5 17 43 9 10 37 20

Lösung: 2 9 5 14 10 20 17 43 89 56 37 33

3. (3 Punkte) Eine Liste wird mit HeapSort sortiert. Die Zeile zeigt die in einen Heap umgewandelte Liste. Füge für die beiden folgenden Reorganisationen je eine Zeile hinzu.

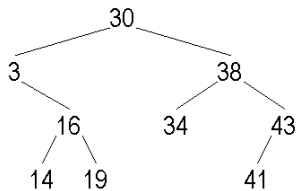
7 12 11 13 47 42 82 36

Lösung:

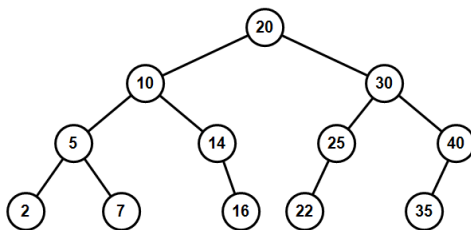
```
11 12 36 13 47 42 82 7
12 13 36 82 47 42 11 7
```

4. (3 Punkte) In einen binären Suchbaum werden nacheinander die Zahlen 30 38 34 3 16 19 43 41 14 eingefügt. Zeichne den Suchbaum.

Lösung:



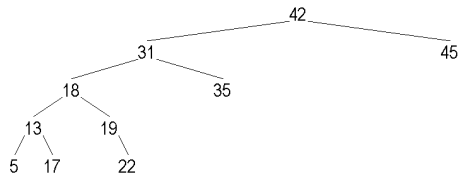
5. (2 Punkte) Gib eine Reihenfolge der Zahlen an, so dass bei der Eingabe der Zahlen in dieser Reihenfolge der abgebildete Suchbaum entsteht.



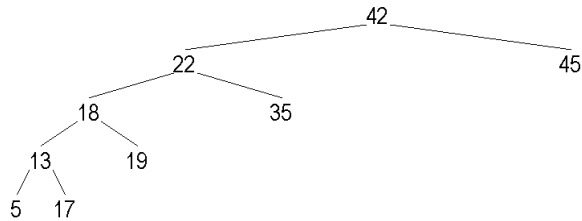
Lösung:

20, 10, 5, 2, 7, 14, 16, 30, 25, 22, 40, 35

6. (2 Punkte) In dem folgenden Suchbaum wird die 31 gelöscht. Zeichne den Suchbaum nach dem Löschvorgang.



Lösung:



7. (3 Punkte) Gegeben sei die Hashfunktion $f(x) = x \% 10$. Füge die Zahlenfolge 27 8 17 16 26 20 37 in die Hashtabelle ein. Benutze dabei quadratisches Sondieren.

f(x)	0	1	2	3	4	5	6	7	8	9
x										

Lösung:

0	1	2	3	4	5	6	7	8	9
26	17	.	37	20	.	16	27	8	.

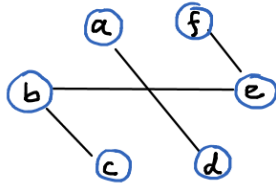
8. (3 Punkte) Gegeben seien die beiden Hashfunktionen $f_1(x) = x \% 10$ und $f_2(x) = (2x + 7) \% 10$. Füge die Zahlenfolge 27 8 17 16 26 19 37 in die Hashtabelle ein. Benutze als Sondierungsverfahren Doublehashing.

f(x)	0	1	2	3	4	5	6	7	8	9
x										

Lösung:

0	1	2	3	4	5	6	7	8	9
37	.	.	.	19	26	16	27	8	17

9. (3 Punkte) Gib für den abgebildeten Graphen die Implementierung in Python als Adjazenzmatrix an. Die Knoten a,b,c,.. sind auf die Indizes 0,1,2,... abgebildet.

**Lösung:**

```

G = [[0,0,0,1,0,0],
      [0,0,1,0,1,0],
      [0,1,0,0,0,0],
      [1,0,0,0,0,0],
      [0,1,0,0,0,1],
      [0,0,0,0,1,0]]

```

10. (3 Punkte) In der Variablen `graph` sei ein ungewichteter Graph als Adjazenzmatrix implementiert. Die Knoten a,b,c,... sind auf die Indizes 0,1,2,... abgebildet. Schreibe eine Anweisung, die in der Variablen `nb` eine Liste mit allen Buchstaben speichert, die Nachbarn von d sind. Hinweis: die Funktion `chr(y+97)` ordnet den Zahlen `y=0,1,2,...` die Buchstaben a,b,c, ... zu.

Lösung:

```
nb = [chr(y+97) for y in range(len(graph)) if graph[3][y] == 1]
```

11. (4 Punkte) Der gewichtete Graph G ist mit einer Adjazenzmatrix implementiert. Die Knoten a,b,c,.. sind auf die Indizes 0,1,2,... abgebildet. Der Floyd-Algorithmus berechnet die abgebildeten Kosten- und Wegematrizen. Ermittle die Länge des kürzesten Weges von b nach e und gib die Länge der Teilstrecken an.

(Der Punkt steht für ∞)

Berechnete Kostenmatrix:

```

0  2  2  3  4
1  0  3  4  5
11 13 0  1  2
.  .  .  0  1
.  .  .  .  0

```

Berechnete Wegematrix:

```

0  0  0  2  3
1  1  0  2  3
2  0  2  2  3
3  3  3  3  3
4  4  4  4  4

```

Lösung:

```

Kürzester Weg von b nach e
(Kosten der Teilstrecken in Klammern)
b - a (1) - c (2) - d (1) - e (1)
Gesamtkosten = 5

```

12. (4 Punkte) Berechne zu der Ausgangsmatrix D^{-1} des Floyd-Algorithmus die Matrix D^3

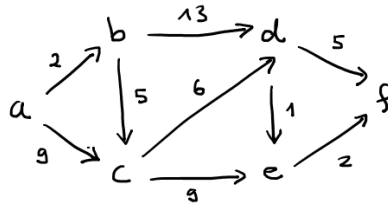
Ausgangsmatrix: (Der Punkt steht für ∞)

0	.	.	1	3
.	0	4	.	12
.	.	0	5	.
.	2	.	0	2
6	.	.	.	0

Lösung:

0	3	7	1	3
.	0	4	9	11
.	7	0	5	7
.	2	6	0	2
6	9	13	7	0

13. (4 Punkte) a. Gib für den abgebildeten Graphen die im Unterricht vorgeschlagene Implementierung in Python an.
 b. Mit welcher Schleife können wir alle Nachbarn des Knoten 'c' durchlaufen?
 c. Welcher boolesche Ausdruck überprüft, ob es eine Kante von 'c' nach 'd' gibt?
 d. Durch welchen Ausdruck kommen wir an das Gewicht der Kante von 'b' nach 'd'?
 Die Antworten für b. c. und d. sollen auch für andere Ausprägungen des Graphen gelten.



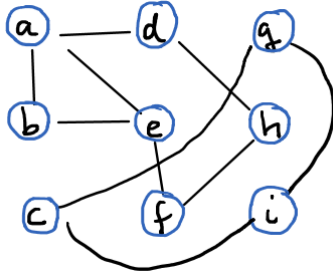
Lösung:

```

a. G = {
    'a': {'b':2, 'c':9},
    'b': {'c':5, 'd':13},
    'c': {'d':6, 'e':9},
    'd': {'e':1, 'f':5},
    'e': {'f':2},
    'f': {}
}

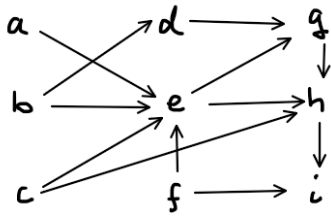
b. for v in G['c']:
    # do something with v
c. 'd' in G['c']
d. G['b']['d']
  
```

14. (3 Punkte) Der Graph wird von Startknoten a aus mit Tiefensuche traversiert. Die Nachbarn werden in alphabetischer Reihenfolge besucht. Ermittle die previsit/postvisit Nummern für jeden Knoten.

**Lösung:**

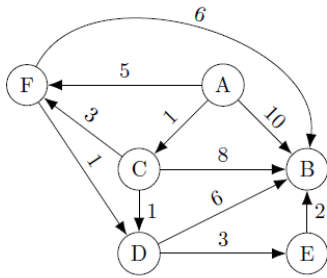
a	1	12
b	2	11
e	3	10
f	4	9
h	5	8
d	6	7
c	13	18
g	14	17
i	15	16

15. (4 Punkte) Der Graph wird mit dem im Unterricht vorgestellten Algorithmus topologisch sortiert. Welche Reihenfolge ergibt sich?

**Lösung:**

f c b d a e g h i

16. (4 Punkte) Ermittle mit dem Algorithmus von Dijkstra die kürzesten Wege von Knoten A zu allen anderen Knoten. Notiere die Reihenfolge der endgültig markierten Knoten. Notiere für jeden Knoten die Reihenfolge der Werte, mit denen er markiert wird.

**Lösung:**

Reihenfolge der endgültigen Markierungen: a c d f e b

a : 0

b : inf 10 9 8 7

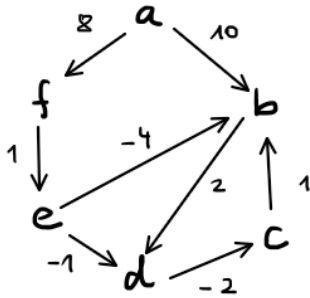
c : inf 1

d : inf 2

e : inf 5

f : inf 5 4

17. (4 Punkte) Führe den Algorithmus von Bellman-Ford mit Startknoten a aus. Notiere nach jedem Durchgang die Kosten an den Knoten

**Lösung:**

	a	b	c	d	e	f
0 :	0	inf	inf	inf	inf	inf
1 :	0	10	10	12	9	8
2 :	0	5	10	8	9	8
3 :	0	5	5	7	9	8
4 :	0	5	5	7	9	8

Aufgabe:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	Summe:
Punkte:	2	2	3	3	2	2	3	3	3	3	4	4	4	3	4	4	4	53